

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №4

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 3, 2)
Руководитель:
Оценка:
Дата сдачи

Оглавление

Задание	2
Внешняя спецификация	3
Листинг программы	4
Распечатка тестов к программе и результатов	7
Программа №1	7

Задание

Дано k литерных строк. Каждая строка содержит латинские и русские буквы, цифры, а также все возможные разделители. Требуется:

1. Выделить из каждой строки и напечатать подстроки, ограниченные с обеих сторон одной или несколькими точками.
2. Среди выделенных подстрок найти подстроку (если таких подстрок несколько, выбирается первая из них), содержащую цифры и круглые скобки.
3. Преобразовать исходную строку, которой принадлежит найденная подстрока, следующим образом, удалить путем сдвига все символы, отличные от русских букв.

Внешняя спецификация

Лабораторная работа №4

Задание №1

Введите k от 1 до $ARRAY_SIZE$:
< k >

★

До $1 \leq k \leq ARRAY_SIZE$

Введите << k >> строк длины не более $STRING_SIZE$:
< $S[0]$ >, < $S[1]$ >, ..., < $S[k-1]$ >

При $n = 0$

Не нашлось подстрок, ограниченных точками

Иначе

Нашлось << n >> подстрок, ограниченных точками:
<< $substrs[0]$ >>, << $substrs[1]$ >>, ..., << $substrs[n-1]$ >>

При $found.size = 0$

Не нашлось строки со скобками и цифрами

Иначе

Нашлась строка со скобками и цифрами: "<< $found$ >>"
После очистки букв не из русского алфавита: "<< $found$ >>"

Листинг программы

```
1 #include <stddef.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 constexpr size_t STRING_SIZE = 1024;
6 constexpr size_t ARRAY_SIZE = 1024;
7
8 typedef struct String
9 {
10     const char* data;
11     size_t size;
12 } String;
13
14 typedef struct String_array
15 {
16     String data[ARRAY_SIZE];
17     size_t size;
18 } String_array;
19
20 static String_array
21 find_all_substrings(size_t k, const char strings[ARRAY_SIZE][STRING_SIZE]);
22 static String_array find_substrings(const char* string);
23 static String find_substring_with_brackets_and_digits(const String_array* subtrs);
24 static bool check_string(String string);
25 static void clean_non_cyrillic(String* string);
26
27 static void print_string(String str);
28 static size_t input_k(void);
29 static void clean_buffer(void);
30
31 int main()
32 {
33     size_t k = input_k();
34
35     printf("Введите %zu строк длины не более %zu:\n", k, STRING_SIZE - 1);
36     char strings[ARRAY_SIZE][STRING_SIZE] = {};
37
38     for (size_t i = 0; i < k; i++)
39     {
40         fgets(strings[i], STRING_SIZE, stdin);
41     }
42
43     String_array subtrs = find_all_substrings(k, strings);
44     if (subtrs.size)
45     {
46         printf("Нашлось %zu подстрок, ограниченных точками:\n", subtrs.size);
47         for (size_t i = 0; i < subtrs.size; i++)
48         {
49             print_string(subtrs.data[i]);
50             printf("\n");
51         }
52     }
53     else
54     {
55         printf("Не нашлось подстрок, ограниченных точками");
56     }
57
58     String found = find_substring_with_brackets_and_digits(&subtrs);
59
60     if (found.size)
61     {
62         printf("Нашлась строка со скобками и цифрами: \n");
63         print_string(found);
64         printf("\n\n");
65
66         clean_non_cyrillic(&found);
67         printf("После очистки букв не из русского алфавита: \n");
68         print_string(found);
69         printf("\n\n");
70     }
71     else
72     {
73         printf("Не нашлось строки со скобками и цифрами\n");
74     }
75 }
```

```

74     }
75 }
76
77 static String_array
78 find_all_substrings(size_t k, const char strings[ARRAY_SIZE][STRING_SIZE])
79 {
80     String_array result = {};
81
82     for (size_t i = 0; i < k; i++)
83     {
84         String_array sub_res = find_substrings(strings[i]);
85
86         for (size_t j = 0; j < sub_res.size; j++)
87         {
88             result.data[result.size++] = sub_res.data[j];
89         }
90     }
91
92     return result;
93 }
94
95 static String_array find_substrings(const char* string)
96 {
97     String_array result = {};
98     size_t len = strlen(string);
99
100     const char* start = strchr(string, '.');
101     if (!start)
102     {
103         return result;
104     }
105
106     while (start < string + len)
107     {
108         while (start < string + len && *start == '.')
109         {
110             start++;
111         }
112         const char* end = strchr(start, '.');
113
114         if (!end)
115         {
116             break;
117         }
118
119         result.data[result.size++] = (String) {start, end - start};
120         start = end;
121     }
122
123     return result;
124 }
125
126 static String find_substring_with_brackets_and_digits(const String_array* substrs)
127 {
128     for (size_t i = 0; i < substrs->size; i++)
129     {
130         if (check_string(substrs->data[i]))
131         {
132             return substrs->data[i];
133         }
134     }
135     return (String){};
136 }
137
138 static bool check_string(String string)
139 {
140     bool has_brackets = memchr(string.data, '(', string.size)
141         && memchr(string.data, ')', string.size);
142     if (!has_brackets)
143     {
144         return false;
145     }
146
147     for (char digit = '0'; digit <= '9'; digit++)
148     {
149         if (memchr(string.data, digit, string.size))

```

```

150     {
151         return true;
152     }
153 }
154
155 return false;
156 }
157
158 static void clean_non_cyrillic(String* string)
159 {
160     const char* read = string->data;
161     char* write = (char*)read;
162     size_t new_size = string->size;
163
164     while (read < string->data + string->size)
165     {
166         if (*read & (1 << 8))
167         {
168             *(write++) = *read;
169         }
170         else
171         {
172             new_size--;
173         }
174         read++;
175     }
176     *write = '\0';
177     string->size = new_size;
178 }
179
180 static void print_string(String str)
181 {
182     for (size_t i = 0; i < str.size; i++)
183     {
184         fputc(str.data[i], stdout);
185     }
186 }
187
188 static void clean_buffer(void)
189 {
190     while (getchar() != '\n')
191         ;
192 }
193
194 static size_t input_k(void)
195 {
196     size_t k = 0;
197     do
198     {
199         printf("Введите k от 1 до %zu:\n", ARRAY_SIZE);
200         scanf("%zu", &k);
201         clean_buffer();
202     } while (!(1 <= k && k <= ARRAY_SIZE));
203
204     return k;
205 }

```

Распечатка тестов к программе и результатов

Программа №1

№	Исходные данные	Результат
1	$X_1 = 1, X_2 = 2, X_3 = 3, Y_1 = 1, Y_2 = 2, Y_3 = 3, x = 2.3$	2.3
2	$X_1 = 0, X_2 = 0.5, X_3 = 1, Y_1 = 0, Y_2 = 0.479425538604, Y_3 = 0.841470984808, x = 0$	0
3	$X_1 = 0, X_2 = 0.5, X_3 = 1, Y_1 = 0, Y_2 = 0.479425538604, Y_3 = 0.841470984808, x = 0.5$	0.479426
4	$X_1 = 0, X_2 = 0.5, X_3 = 1, Y_1 = 0, Y_2 = 0.479425538604, Y_3 = 0.841470984808, x = 1$	0.841471
5	$X_1 = 0, X_2 = 0.5, X_3 = 1, Y_1 = 0, Y_2 = 0.479425538604, Y_3 = 0.841470984808, x = 0.7$	0.624244